Deep Reinforcement Learning

Forest Agostinelli
University of South Carolina

Outline

- Logistics
- Context in the field of AI
- Reinforcement learning
- Deep neural networks
- Search
- Open problems

Faculty and Staff

- Instructor
  - Forest Agostinelli
    - B.S. in Electrical and Computer Engineering from Ohio State
    - M.S. in Computer Science from the University of Michigan
    - Ph.D. in Computer Science from the University of California, Irvine
  - Area of research
    - Solving pathfinding problems with deep learning, reinforcement learning, heuristic search, and logic
    - Explainable AI
    - Applications of AI to the sciences
    - Website: https://cse.sc.edu/~foresta/

Course Logistics

- Blackboard will serve as the course website
- Textbook: Reinforcement Learning: An Introduction (2nd Edition)
  - Freely available here: http://incompleteideas.net/book/RLbook2020.pdf
- Office hours
  - After class

Piazza

- For all questions regarding assignments and course concepts, please use Piazza
  - You can also post anonymously, if you wish
- Questions should be conceptual, logistical, or used to report a general bug
- Please do not post code on Piazza!

Homework

- There will be 3-4 coding homework assignments throughout the semester
  - Lowest 1 grade will be dropped
- Homework can be up to 1 day late with a penalty of 20 percentage points
  - i.e., 90% -> 70%
- Deadlines are strict
  - Submit early and submit often
- Coding will be done in Python
  - Students are expected to be proficient coders
  - If you do not know Python you will be expected to learn it independently and rapidly
    - https://www.tutorialspoint.com/python/index.htm
- Coding homework 0 is out!
  - Environment setup
  - Python tutorial
  - Nothing to turn in for homework 0, nevertheless, it is very important!
- **All questions where code does not run will be marked as a 0**

Quizzes

- 4-5 30-minute quizzes to assess your conceptual knowledge
- Lowest score will be dropped
- There will be assigned seating during the quizzes
- **All illegible answers will be marked as a 0**

Midterm

- Will cover the first part of the class
- There will be assigned seating during the midterm
- There will be no final exam and, instead, we there will be a class project

Project

- The class project gives you the opportunity to implement deep reinforcement learning algorithms to solve a problem of your choosing.
- The project will be done in teams of two to three.
- You are encouraged to incorporate aspects of your own research into your project
- If you are not doing a research project, you can do a replication of existing work
  - Standards for replication will be high
- Project stages
  - Proposal – Week 5
  - Progress report – Week 8
  - Presentation – Weeks 14-15
  - Report and code – Finals week

Excused Absences

- All excused absences for homework or quizzes/exams require a letter for the Office of Student Advocacy

Academic Integrity

- Must uphold high academic standards
- Must protect the honest and hard-working students (vast majority)
- All work turned in must be your own. Plagiarism of any kind, including from online sources, is strictly prohibited. All potential Honor Code violations will be reported to the Office of Academic Integrity. **Honor Code violations of any kind (including plagiarism) on the homework or quizzes will result in a zero on that assignment and your lowest grade will no longer be dropped. Honor Code violations of any kind (including plagiarism) on the midterm or project will result in failure of the course.** You can familiarize yourself with the Honor Code here: http://www.sc.edu/policies/ppm/staf625.pdf

Academic Integrity

- The project can contain helper functions written by other people provided that they are cited in the code and in the report.
- They may consist of a small portion of your project that does not implement the main topic of your project
  - Data loading/processing
  - Visualizations
  - Small mathematical functions
- If there are ANY questions please contact me

Grading

- Homework – 20%
- Quizzes – 25%
- Midterm – 25%
- Project – 25%
  - Proposal – 2%
  - Progress report – 3%
  - Presentation – 10%
  - Report and Code – 10%
- Paper presentation – 5%

Special Thanks

- Richard Lathrop
- Pieter Abbeel
- David Silver
- Pooyan Jamshidi
- Emma Brunskill
- Pascal Poupart
- Roy Fox

Getting Involved in Research

- Lots of exciting research going on at USC
  - https://www.sc.edu/study/colleges_schools/engineering_and_computing/departments/computer_science_and_engineering/our_people/index.php
- My own research: heuristic search, deep learning, reinforcement learning, logic, explainable AI, applications

Topics Covered in This Class

- **Part 1: Foundation**
  - Markov decision processes
  - Policy and value functions
- **Part 2: Tabular Reinforcement Learning**
  - Dynamic programming
  - Model-free reinforcement learning
- **Part 3: Deep Reinforcement Learning**
  - Deep neural networks
  - Approximate dynamic programming
  - Policy gradients
- **Part 4: Search**
  - Model-based RL and search
  - A* Search
  - Monte-Carlo Tree Search
- **Part 5: Advanced Topics**
  - Partial observability
  - Meta-learning
  - Explainability
  - Generative models

Outline

- Logistics
- Context in the field of AI
- Reinforcement learning
- Deep neural networks
- Search
- Open problems

How Would You Define Artificial Intelligence?

- How about reinforcement learning?

How I Define Artificial Intelligence

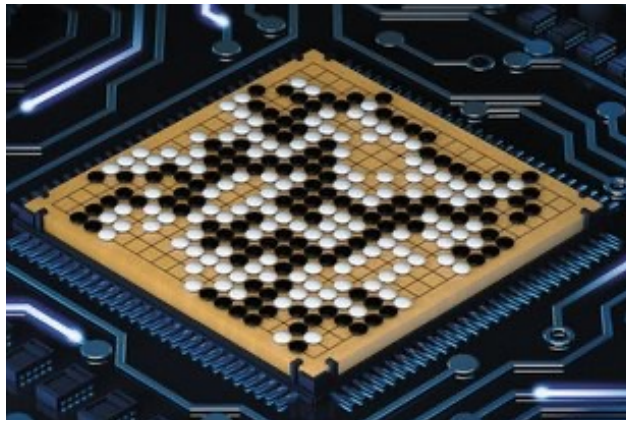
- Artificial intelligence is the study of algorithms that implement algorithms
- An algorithm is a sequence of instructions
  - An algorithm is defined by a specification
  - Algorithms have been fundamental to human advancement and have existed well before the invention of computers
  - Computers have made the execution of certain algorithms quick and precise
  - AI operates at one more level of abstraction by automating the implementation of algorithms that can be executed on a computer
- To implement an algorithm means to find a sequence of instructions that meets a given specification

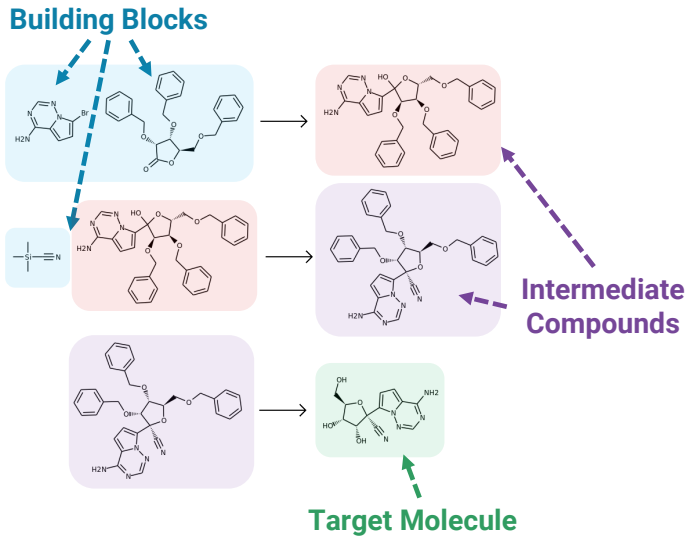
Solving a Problem With AI

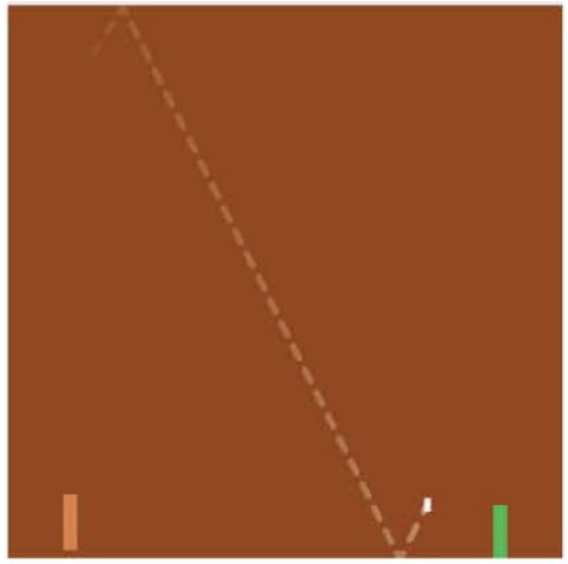
- Given a problem, we would like to use an AI algorithm to implement an algorithm that solves that problem
- We first must formally define the problem
  - What class of problem does it belong to?
- We then find or invent an AI algorithm that is applicable to that class of problems

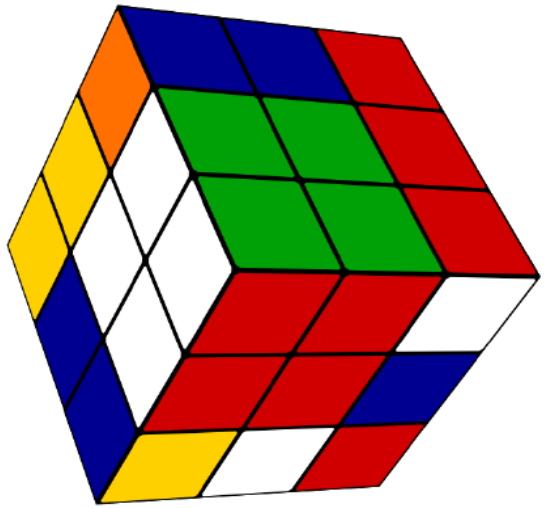
Reinforcement Learning

- Used for solving a class of sequential decision-making problems
  - Learn to make decisions that maximize expected future reward
- Learning is done through experience
- Decisions affect experience and experience affects decisions
- There is often uncertainty involved with this process
  - What happens if I make this decision?
  - How good is this outcome?



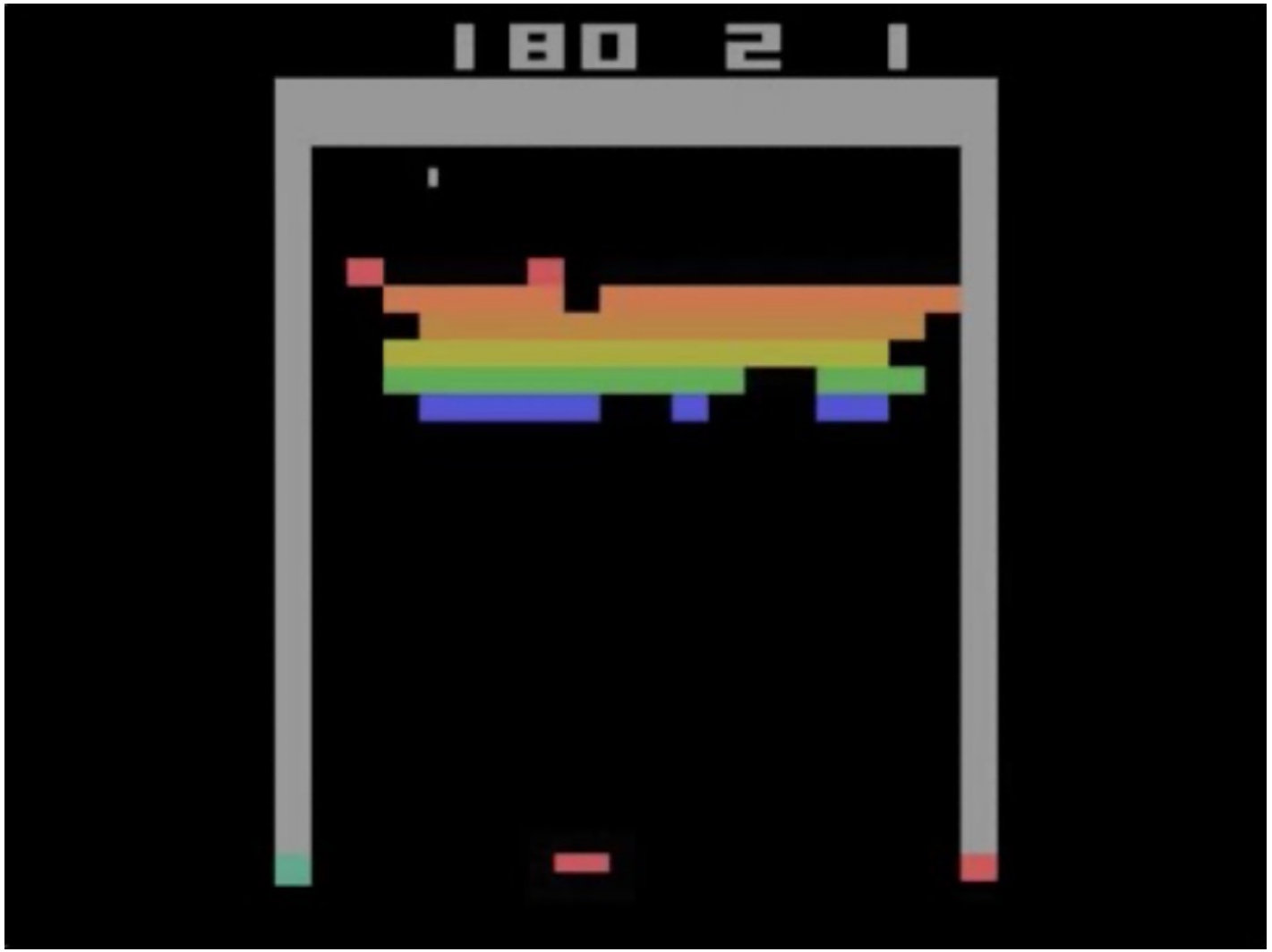






RL Examples: Atari

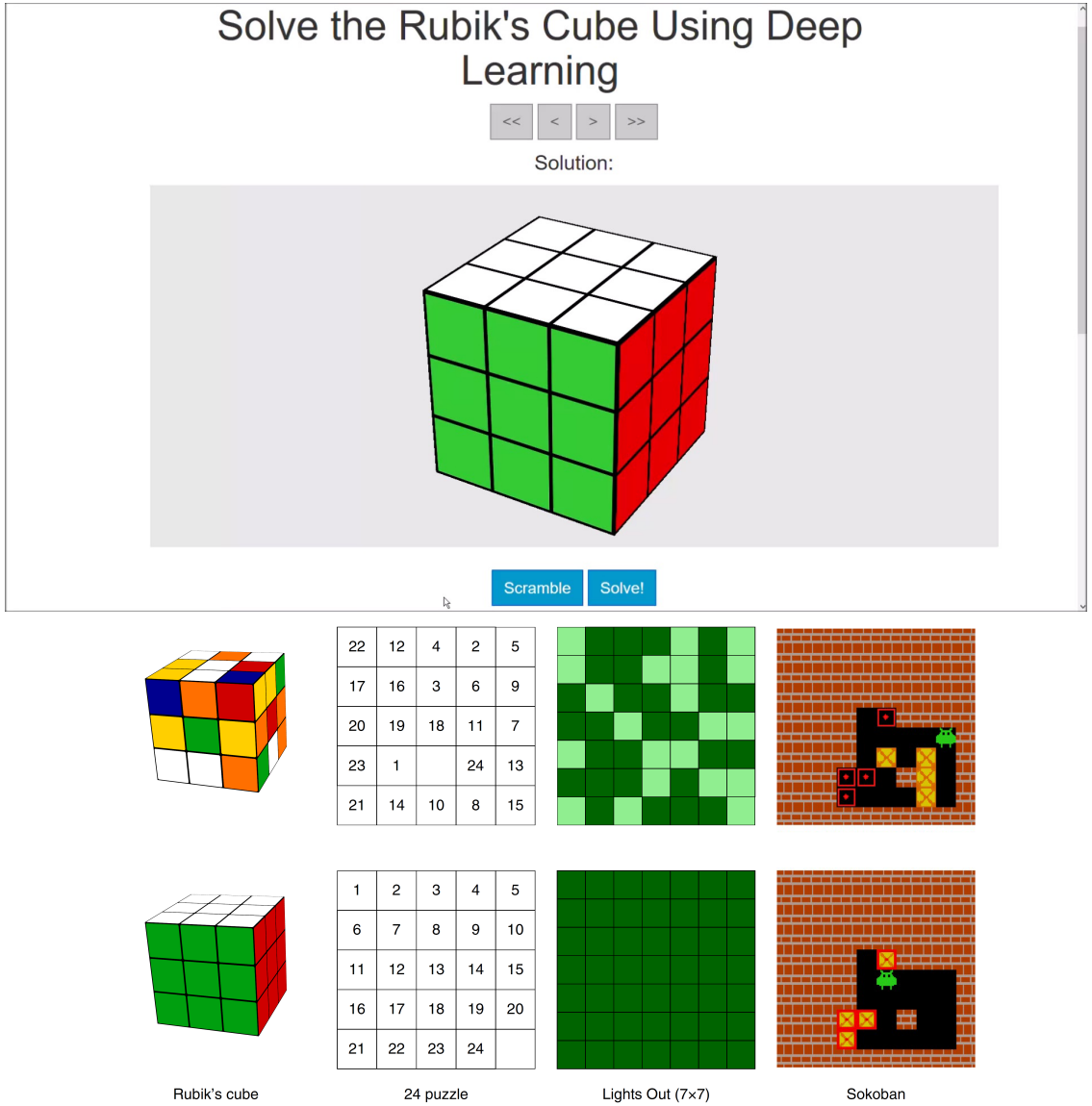
- Using RL algorithms, a deep neural network is trained to play Atari games using raw pixels.
- Current work shows we can train deep neural networks to play better than humans on 57 different Atari games.



Mnih, Volodymyr, et al. "Human-level control through deep reinforcement learning." Nature 518.7540 (2015): 529-533.

RL Examples: Solving the Rubik's Cube

- 4.3 x 10^{19} possible combinations
- No domain-specific knowledge
- DeepCubeA solves the Rubik's cube and other puzzles
  - Puzzles have up to 3.0 x 10^{62} possible combinations.
- Finds a shortest path in the majority of verifiable cases
- http://deepcube.igb.uci.edu/



Agostinelli, Forest, et al. "Solving the Rubik's cube with deep reinforcement learning and search." Nature Machine Intelligence 1.8 (2019): 356-363.

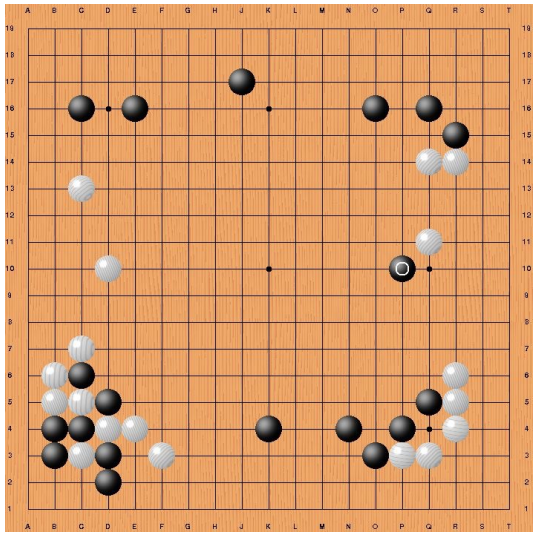
RL Examples: Robotics

- Can solve problems in continuous environments
- Generalizes to cases where there is visual clutter

Levine, Sergey, et al. "End-to-end training of deep visuomotor policies." The Journal of Machine Learning Research 17.1 (2016): 1334-1373.

RL Examples: Go

- AlphaGo learned how to play Go from expert demonstrations data and from self-play
  - Defeated one of the best Go players, Lee Sedol, 4 to 1
  - Move 37
- AlphaGoZero builds on AlphaGo and learns only from self-play



Silver, David, et al. "Mastering the game of go without human knowledge." nature 550.7676 (2017): 354-359.

Deep Reinforcement Learning and Search

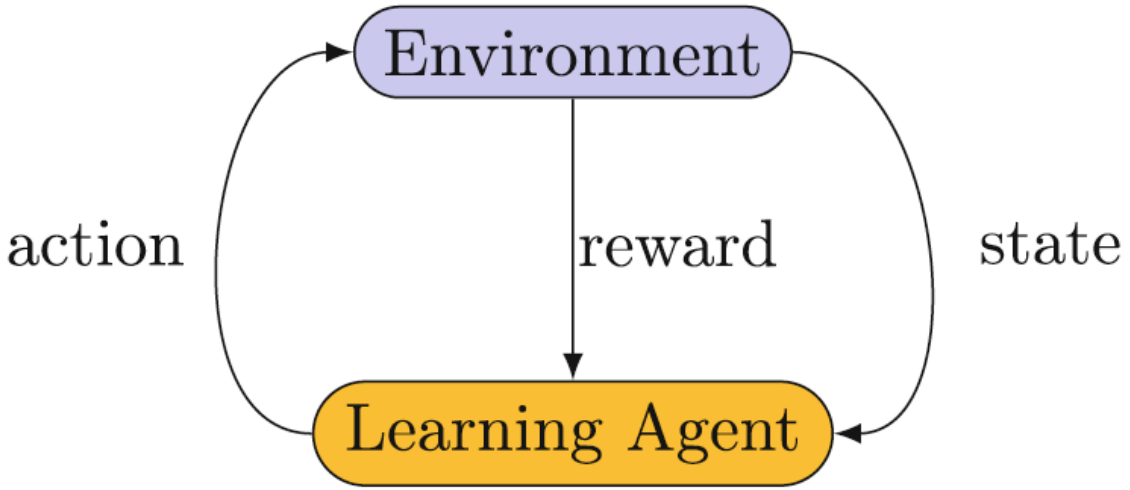
- One of the simplest settings for reinforcement learning is when every configuration of the world, also called a state, fits neatly into a table
  - Easy to work with
  - Theoretical guarantees
- The number of possible states is often very large, sometimes even infinite. Therefore, we need approximation architectures such as deep neural networks
  - The number of parameters is often smaller than the number of states by many orders of magnitude
  - Lose theoretical guarantees, but there are many strategies that work well in practice
- There are almost always non-trivial approximation errors. Therefore, blindly trusting an approximation architecture could lead to significant problems. Therefore, agents must have the ability to search or “think ahead”.
  - Approximation architectures become tools to guide search
  - Can be used to solve complex problems that otherwise would not have been solved

Outline

- Logistics
- Context in the field of AI
- Reinforcement learning
- Deep neural networks
- Search
- Open problems

Reinforcement Learning

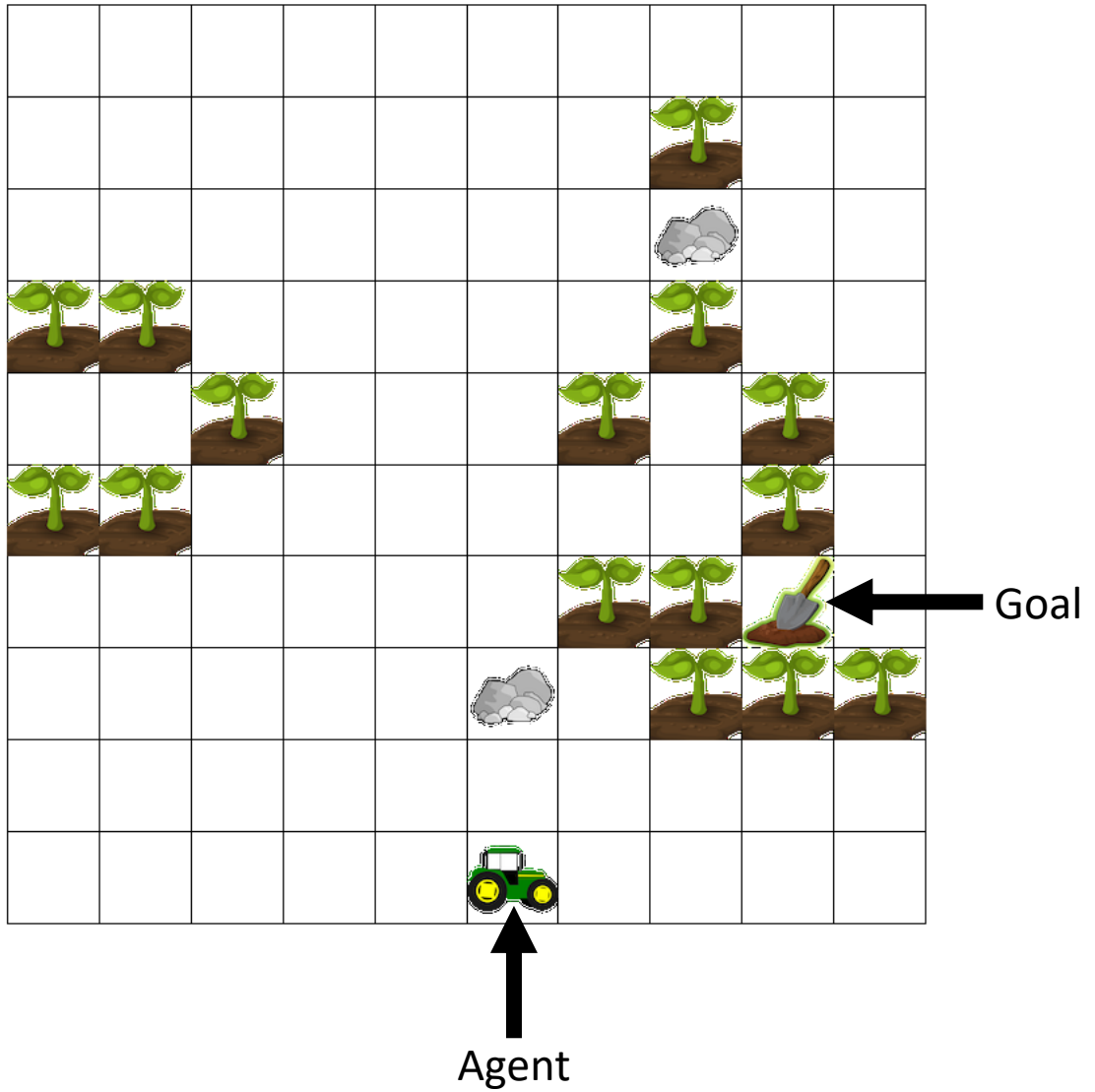
- **Reinforcement learning**: learning to map states to actions so that we maximize the reward we receive from the environment.
- This mapping of states to actions is called a policy function.
  - Deterministic: $a = \pi(s)$
  - Stochastic: $\pi(a|s) = P(A = a|S = s)$
- At each time step t
  - In state S_t, agent takes action A_t
  - Based on state s_t and action a_t, the environment transitions to state S_{t+1} and outputs reward R_{t+1}



Agostinelli, Forest, et al. "From reinforcement learning to deep reinforcement learning: An overview." Braverman Readings in Machine Learning. Key Ideas From Inception to Current State. Springer, Cham, 2018. 298-328.

Reinforcement Learning: A Simple Example

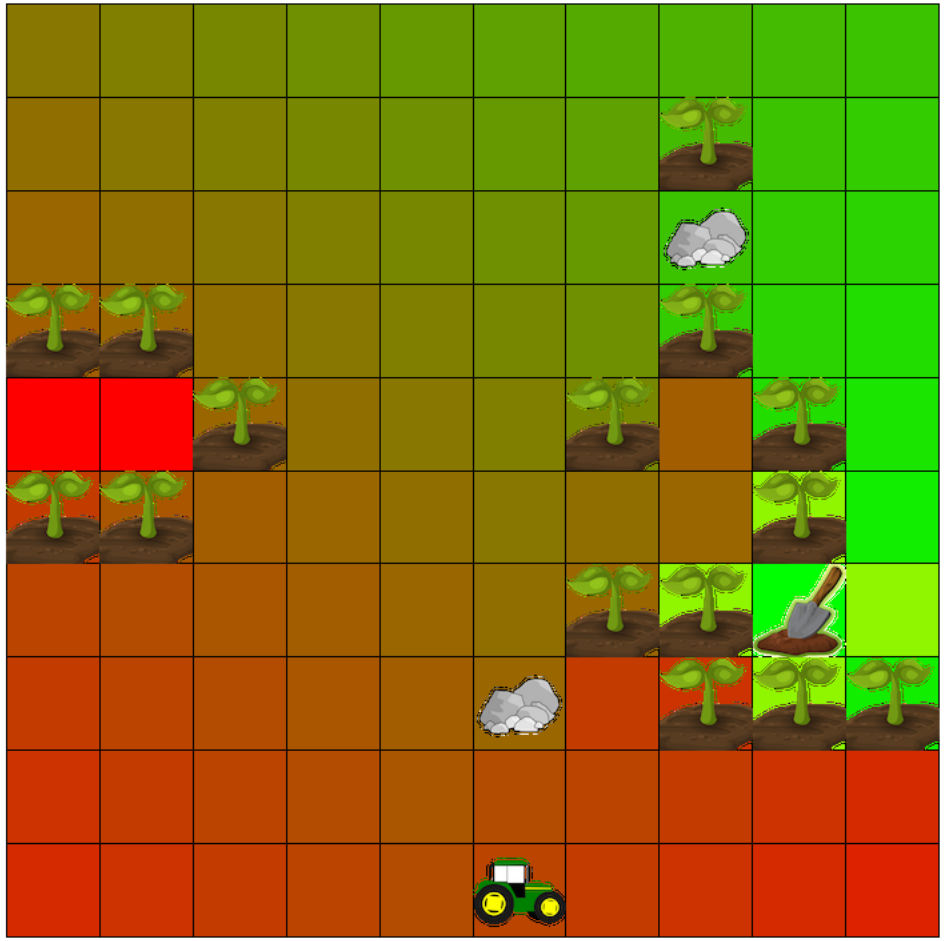
- **State**: the configuration of the environment
  - Location of agent, plants, rocks, and goal
- **Action**: A decision made by the agent that can affect the state of the environment
  - up, down, left, right
- **Reward**: A scalar signal sent from the environment to the agent that is a function of the current state, the action, and the next state
  - Large negative reward for driving on a plant
  - Medium negative reward for driving on a rock
  - Small negative reward for driving on any other space
- What is the optimal policy?
  - The policy that maximizes reward we receive from the environment



Visualization thanks to: https://github.com/nitesh4146/Grid-World-Robot

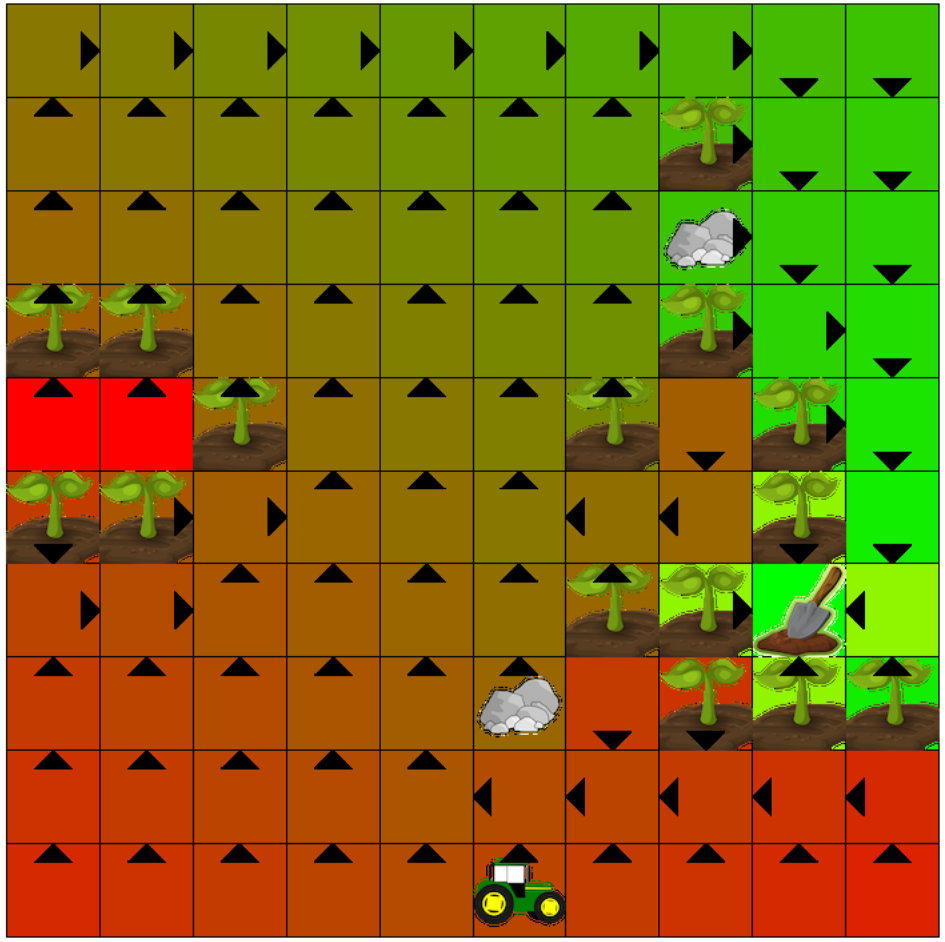
Reinforcement Learning: State-Value Function

- The state-value function predicts the future reward obtained from a given state s when following policy π.
  - $v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots | S_t = s]$
- Using dynamic programming techniques, we can find the optimal state-value function, v^*
  - This is the value function that corresponds to the optimal policy



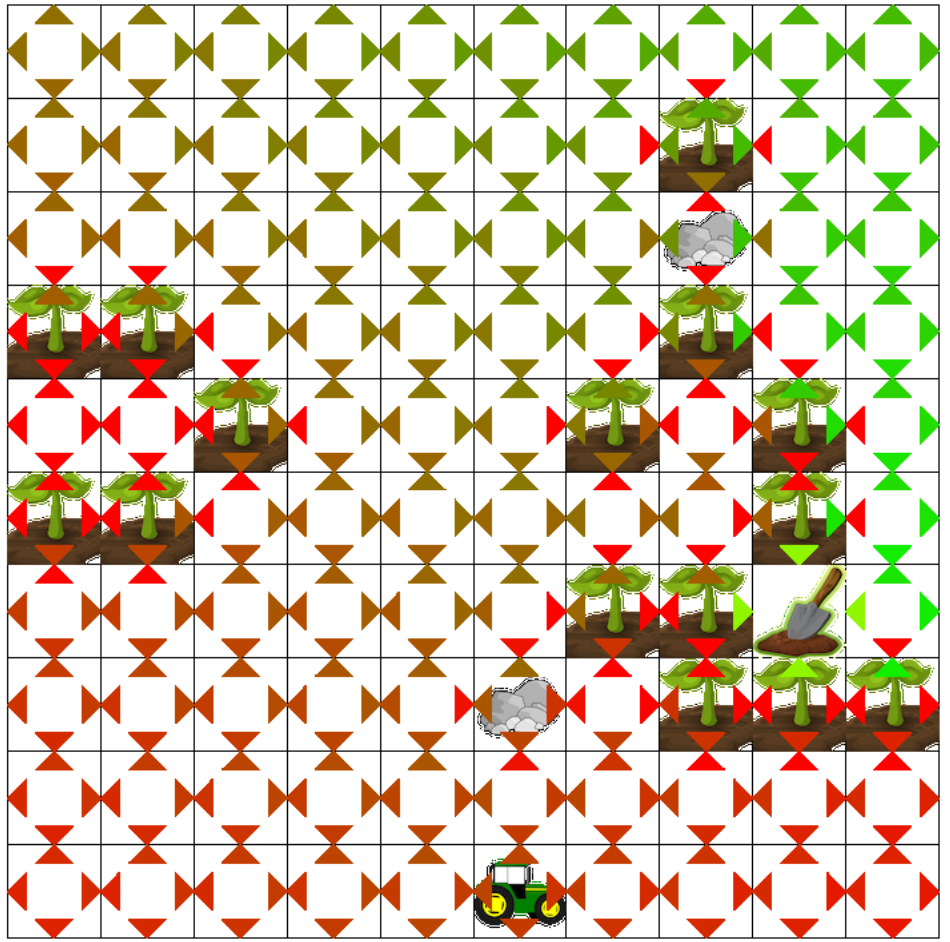
Reinforcement Learning: State-Value Function

- To obtain the optimal policy, we act greedily with respect to the optimal value function
- However, this requires a model of the environment
  - Given a state and an action, we can predict the next state and the rewards



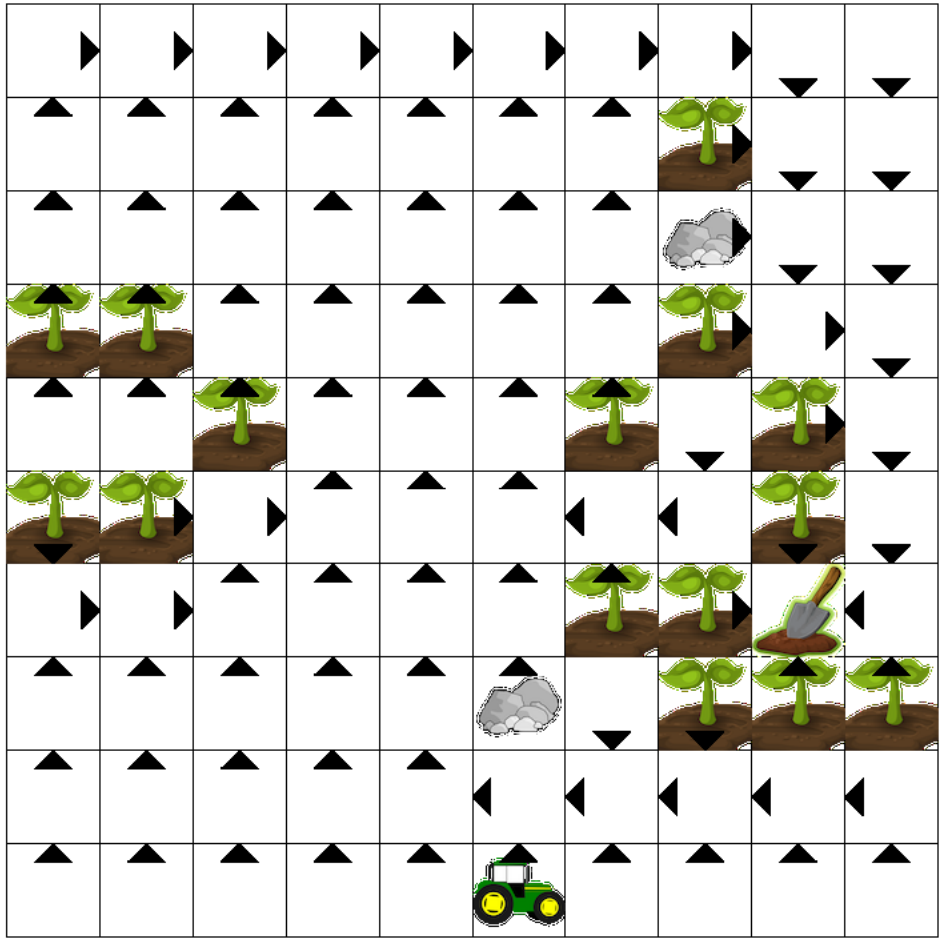
Reinforcement Learning: Action-Value Function

- The action-value function predicts the future reward obtained from a given state s and action a when following policy π.
  - $q_\pi(s,a) = \mathbb{E}_\pi[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots | S_t = s, A_t = a]$
- Using temporal difference methods, we can find the optimal action-value function q_*
- To obtain the optimal policy, we select the action that maximizes the optimal action-value function
  - $a = \underset{a}{\operatorname{argmax}}\, q_*(s,a)$



Reinforcement Learning: Why We Need Deep Learning

- The aforementioned methods represent state-value function, action-value, and policy functions using a table
- We are assuming that we can enumerate every possible state
  - Environments with very large state space?
  - Continuous environments?
  - States with low-level features?
    - Raw pixels instead of discrete locations
- Deep neural networks are powerful function approximators
- We can combine reinforcement learning algorithms with deep learning to learn to solve complex problems

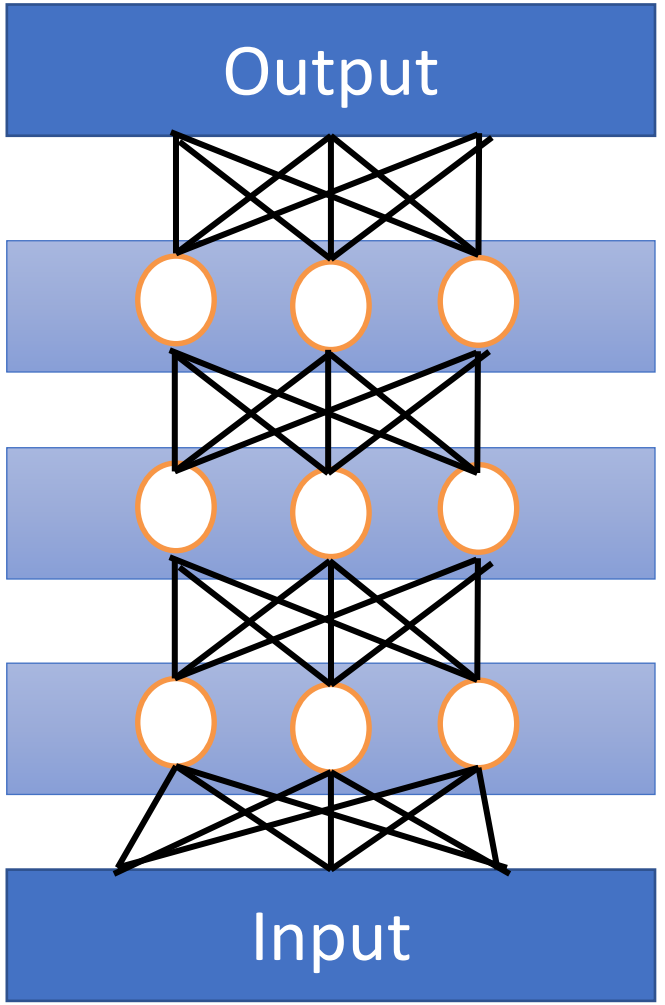


Outline

- Logistics
- Context in the field of AI
- Reinforcement learning
- Deep neural networks
- Search
- Open problems

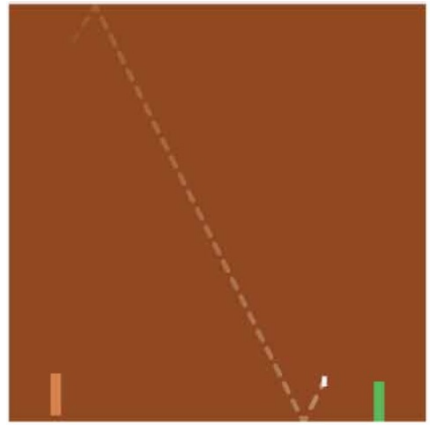
Deep Neural Networks

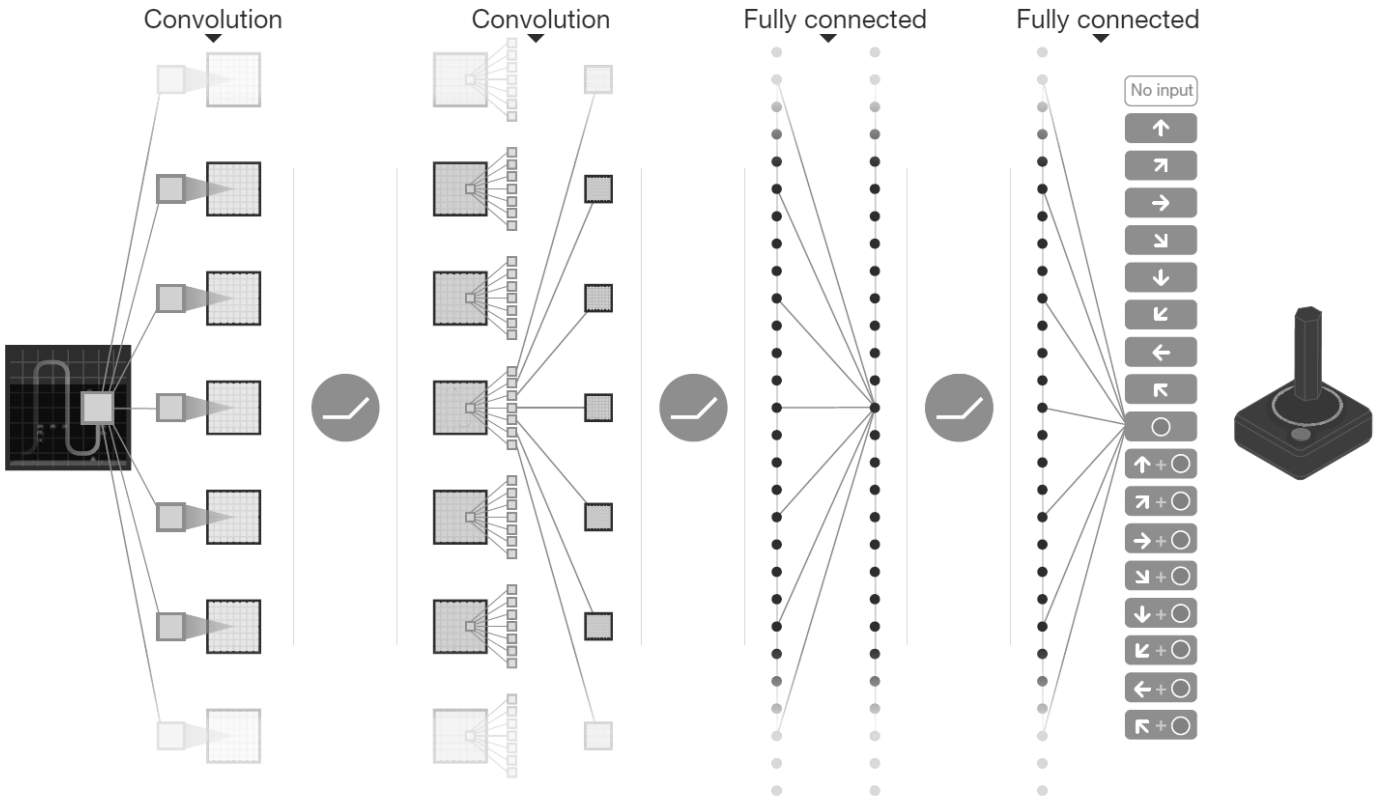
- A class of architectures that can be used for function approximation
  - $y = f(x)$
  - In particular, we are interested in approximating state-value functions, action-value functions, policy functions, and transition functions
- Deep learning algorithms can be used to train these architectures to approximate functions



Deep RL: Atari

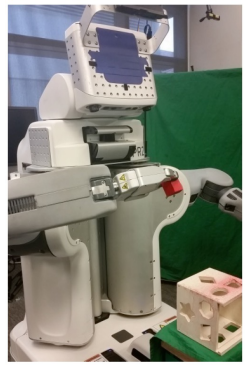
- Used a deep neural network to learn the action-value function, $q_\pi(s, a)$
- Learns only from raw pixels and its own experience

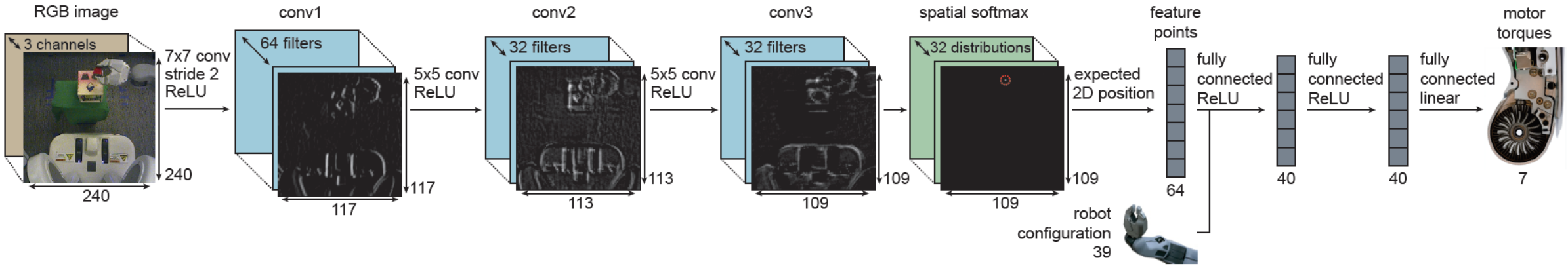




Deep RL: Robotics

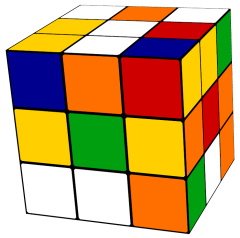
- Used a deep neural network to learn the policy, $\pi(a|s)$



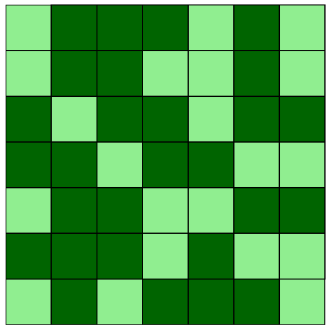


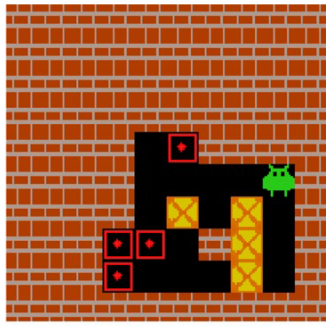
Deep RL: Rubik's Cube

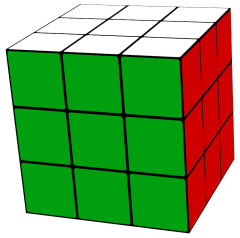
- A deep neural network is used to map a puzzle to its estimate value (negative of the estimated number of steps left to go to solve it)



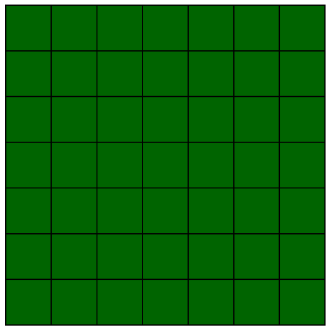
22	12	4	2	5
17	16	3	6	9
20	19	18	11	7
23	1		24	13
21	14	10	8	15

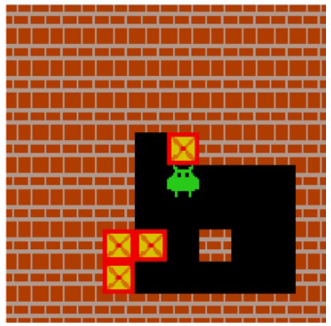






1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	



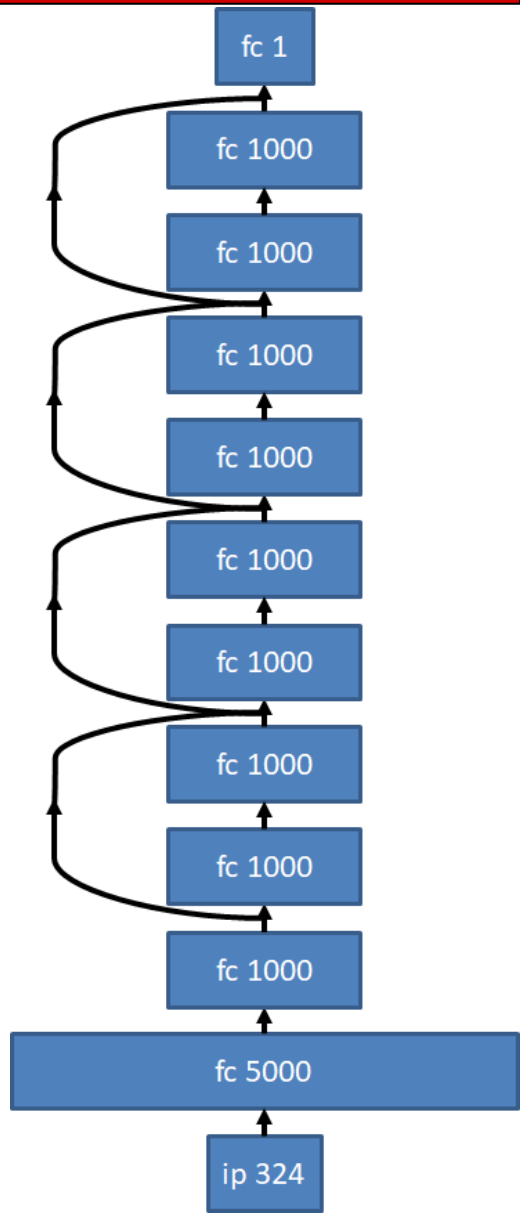


Rubik's cube

24 puzzle

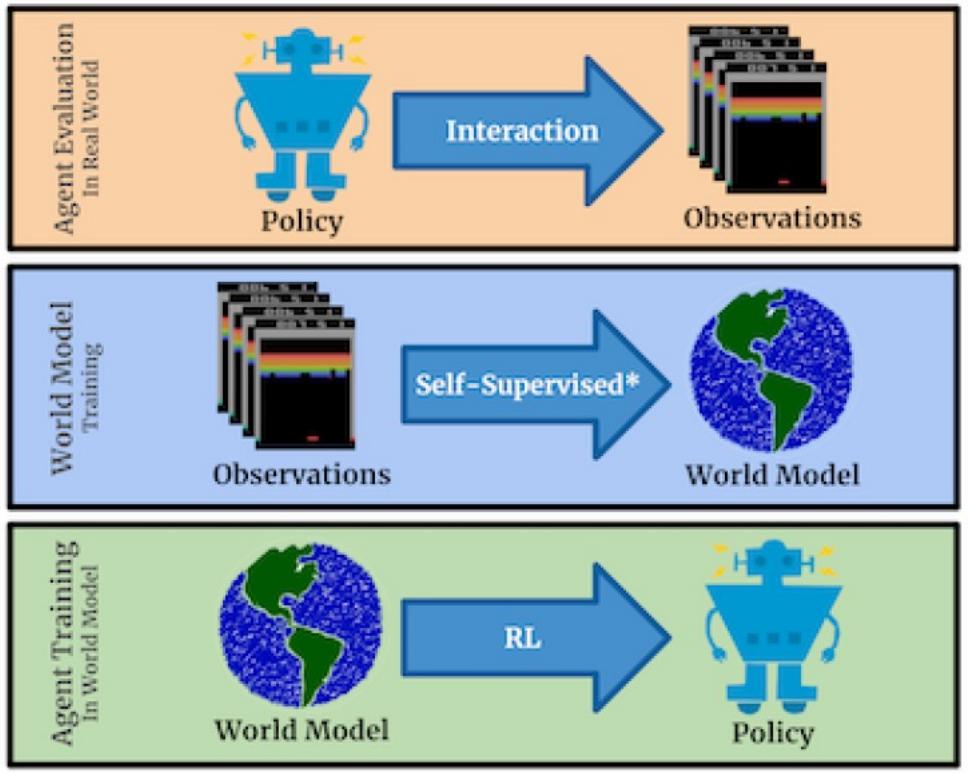
Lights Out (7×7)

Sokoban



Deep Learning: Model-Based RL

- Learning through interacting with the real world can be time consuming or even dangerous
- We can use deep neural networks to learn how the world works
- We can use this model of the world (also referred to as imagination) to train a policy
- Finally, we can use this model to search ("think ahead") when solving a problem



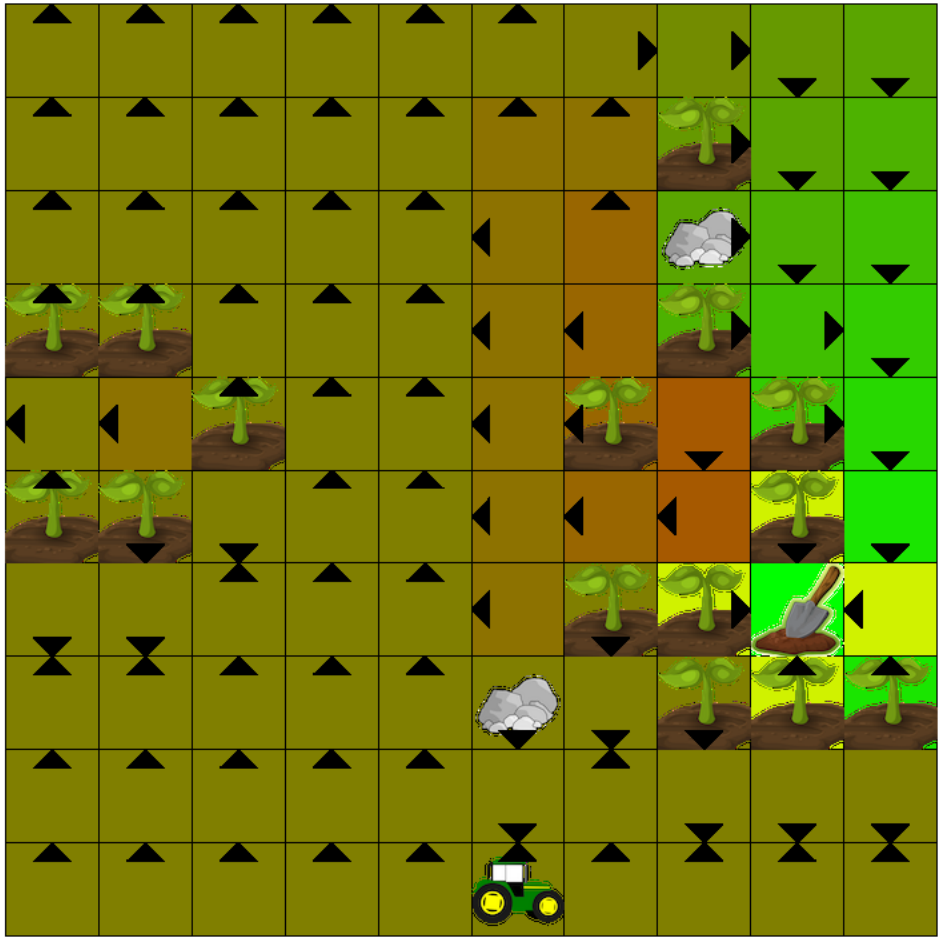
Kaiser, Lukasz, et al. "Model-based reinforcement learning for atari." arXiv preprint arXiv:1903.00374 (2019).

Outline

- Logistics
- Context in the field of AI
- Reinforcement learning
- Deep neural networks
- Search
- Open problems

Deep Learning: Why We Need Search

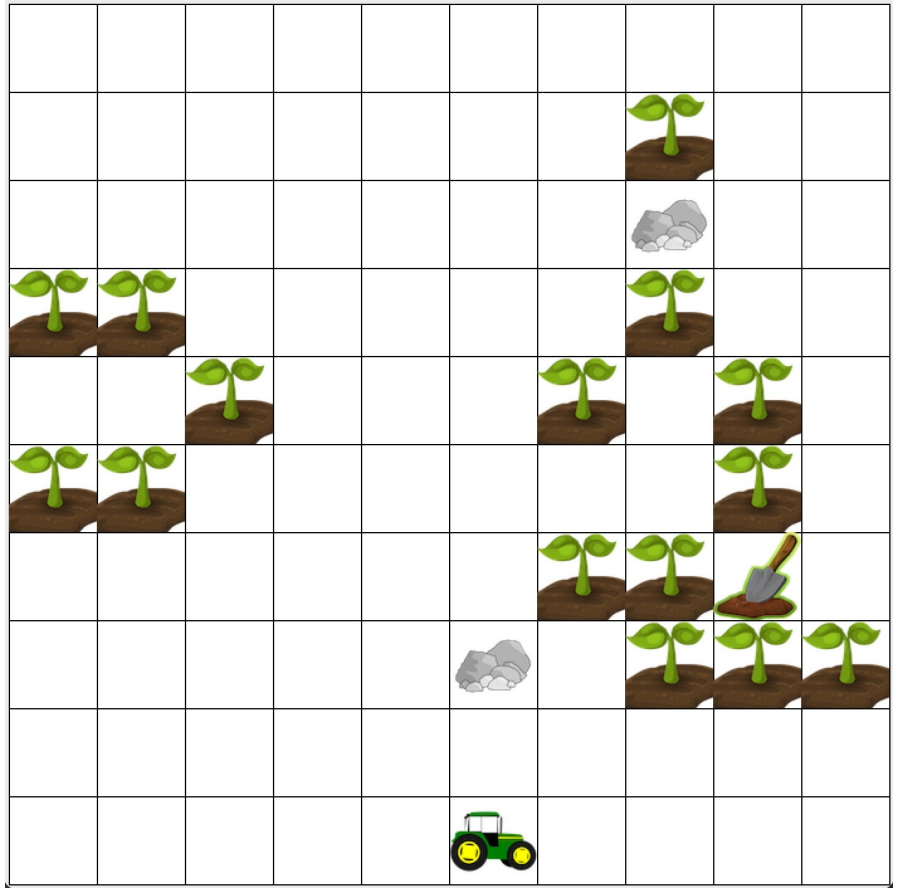
- Deep neural networks are not perfect function estimators
- For some problems, using only your learned policy will not solve the problem
- Search gives machines the ability to think by anticipating the outcomes of various actions
- We can use search to obtain a better policy



Imperfect value function which produces an imperfect policy

Search

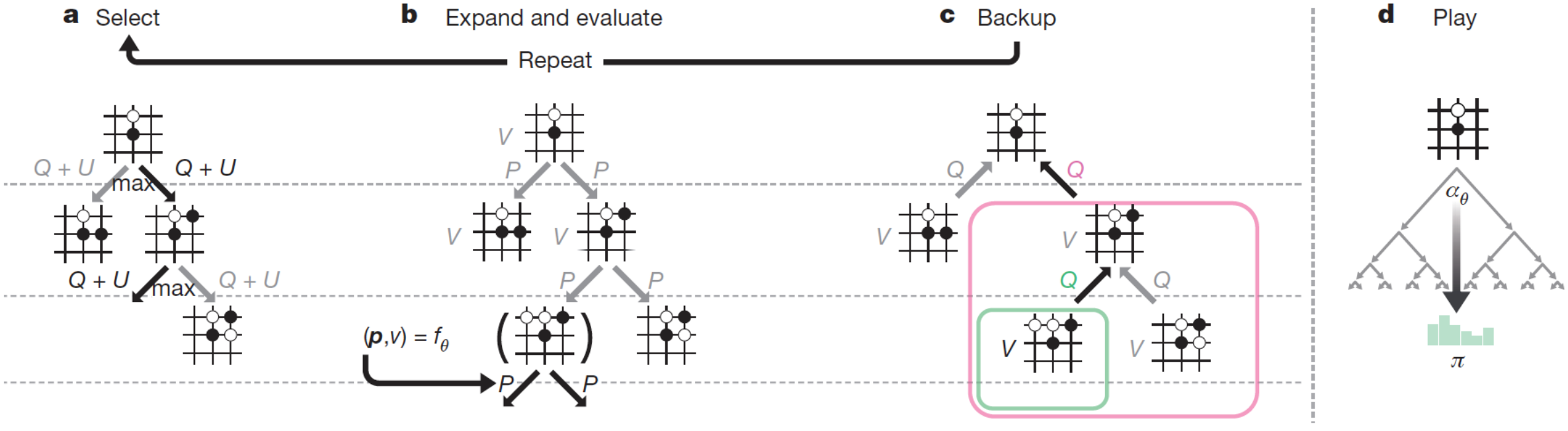
- Search allows us to anticipate the outcomes of our actions
  - Gives the agent the ability to “think” before it acts
- We can use search to prioritize promising paths with pruning paths that not fruitful



By Subh83 - Own work, CC BY 3.0, https://commons.wikimedia.org/w/index.php?curid=14916867

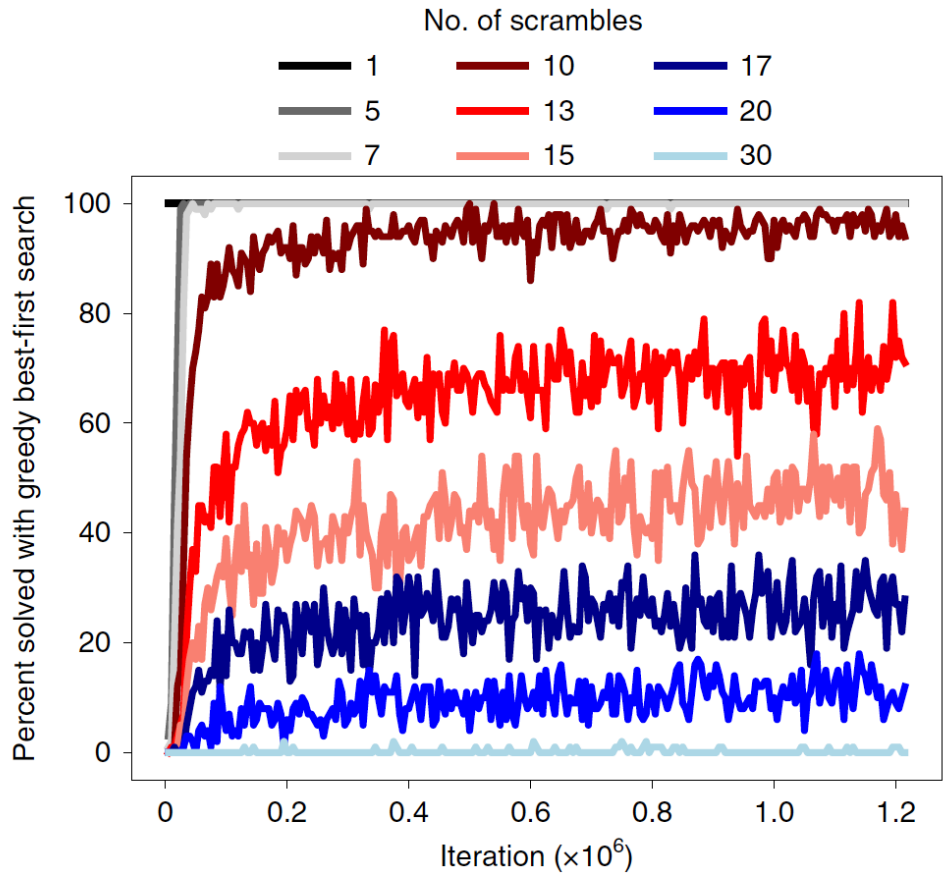
Deep RL and Search: Go

- Used search to obtain a better policy for training and for live gameplay
- Move 37
  - Initial policy assigned low probability to move 37
  - Using search, AlphaGo realized that this was actually a good move



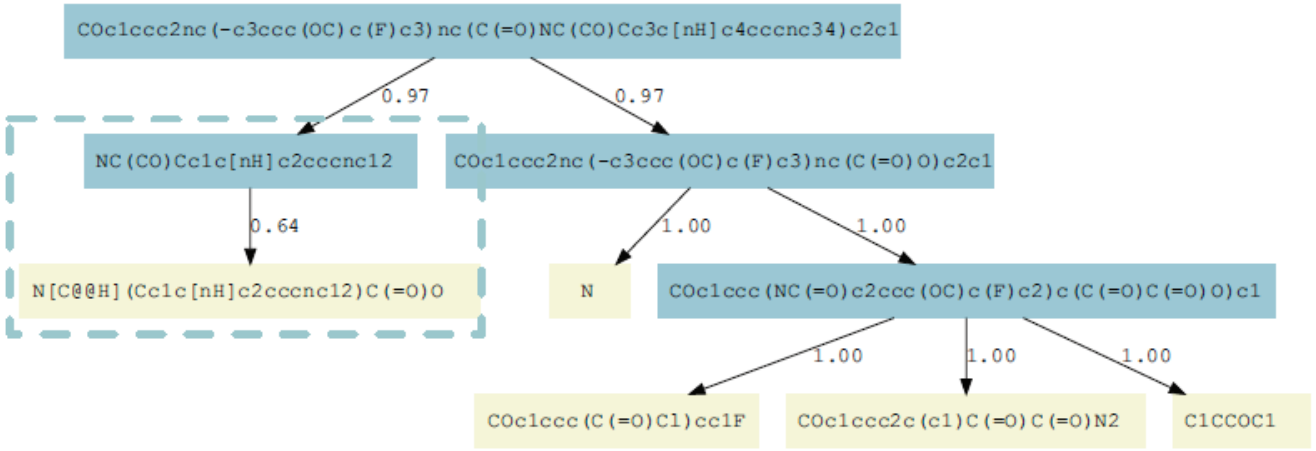
Deep RL and Search: Rubik's Cube

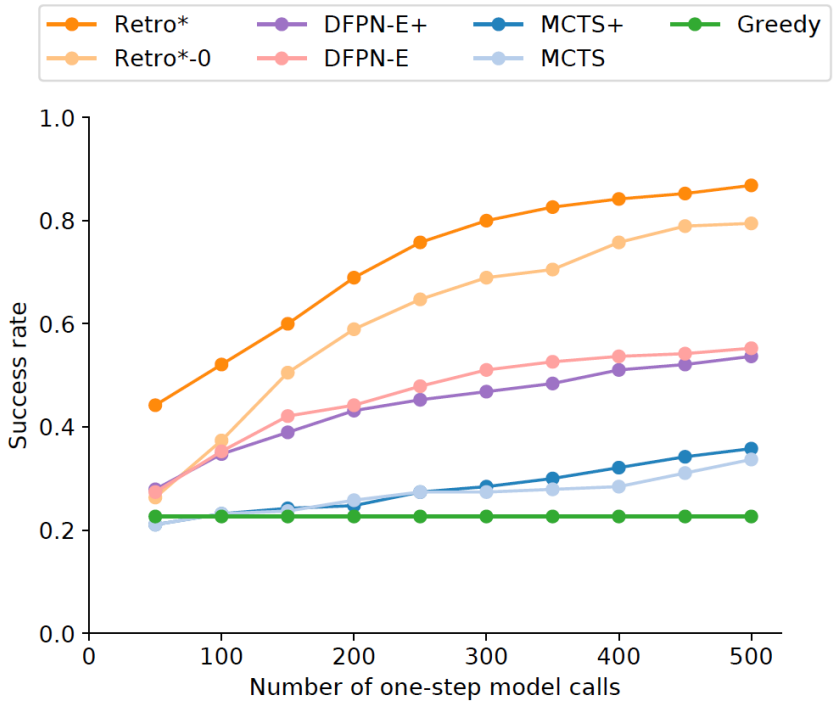
- Without search, the trained value function fails after a certain point
- Combined the value function as a heuristic for A* search
- Solved 100% of puzzles investigated
  - Including the "most difficult" Rubik's cube and 15-puzzle configurations
- Found a shortest path in the majority of verifiable cases



Search: Synthesis of Chemical Compounds

- Retro* used search to find ways to synthesize chemical compounds
- Performs significantly better than a naïve greedy algorithm





Chen, Binghong, et al. "Retro*: Learning Retrosynthetic Planning with Neural Guided A* Search." *ICML* (2020).

Search: Generative Models

- Large language models can produce better responses using search

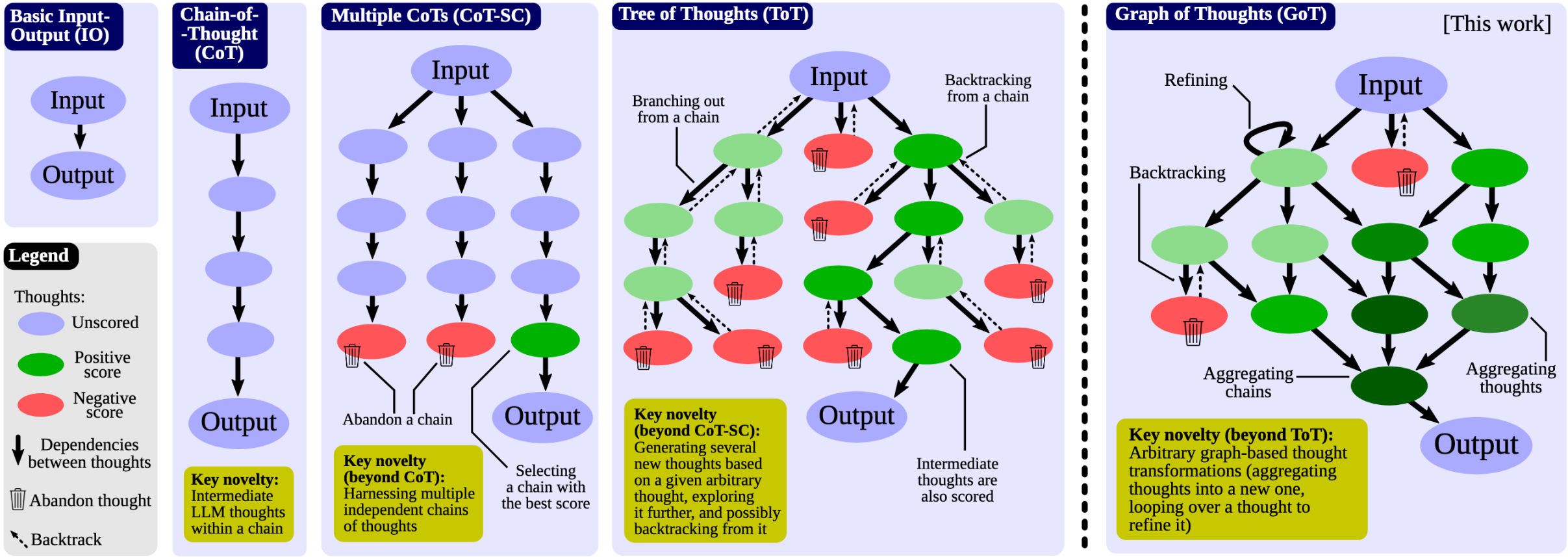


Figure 1: Comparison of Graph of Thoughts (GoT) to other prompting strategies.

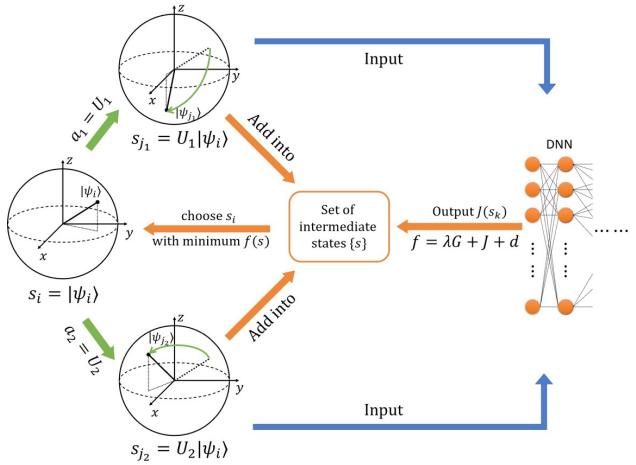
Besta, Maciej, et al. "Graph of thoughts: Solving elaborate problems with large language models." Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 38. No. 16. 2024.

Outline

- Logistics
- Context in the field of AI
- Reinforcement learning
- Deep neural networks
- Search
- Open problems

Open Problems: Connection to the Natural Sciences

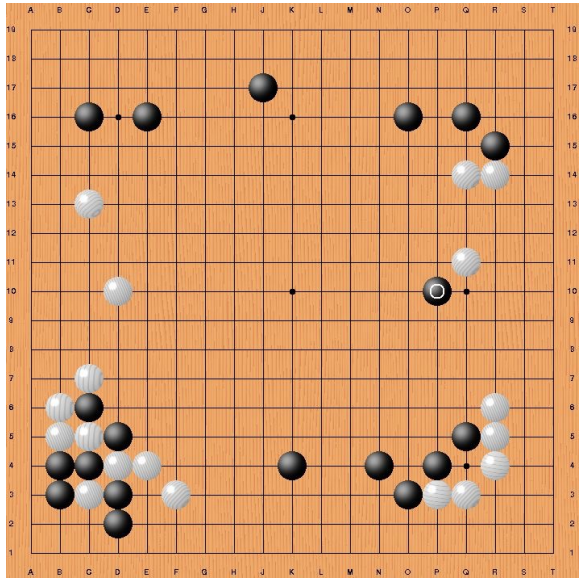
- Many problems in the natural sciences can be posed as reinforcement learning problems
  - Chemistry
  - Quantum computing
  - Bioinformatics
- The correct way to characterize these problems and apply deep reinforcement learning and search techniques is an open problem

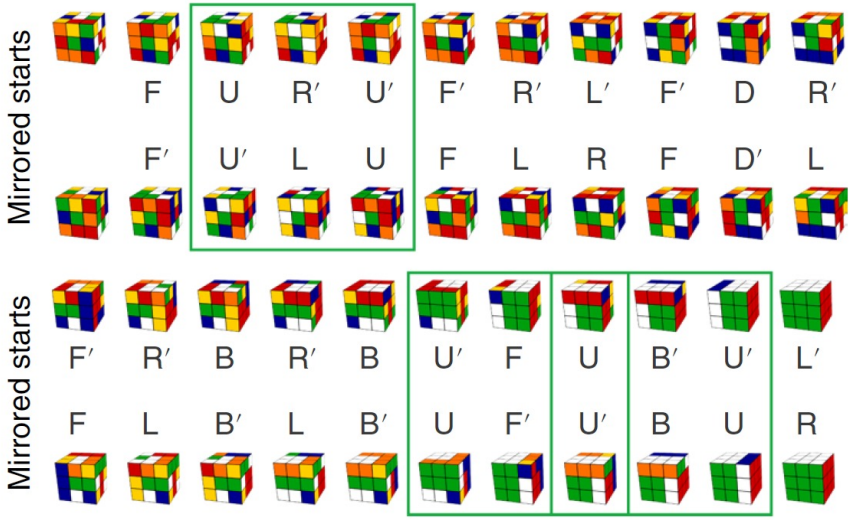


Zhang, Yuan-Hang, et al. "Topological Quantum Compiling with Reinforcement Learning." *arXiv preprint arXiv:2004.04743* (2020).

Open Problems: Explainability

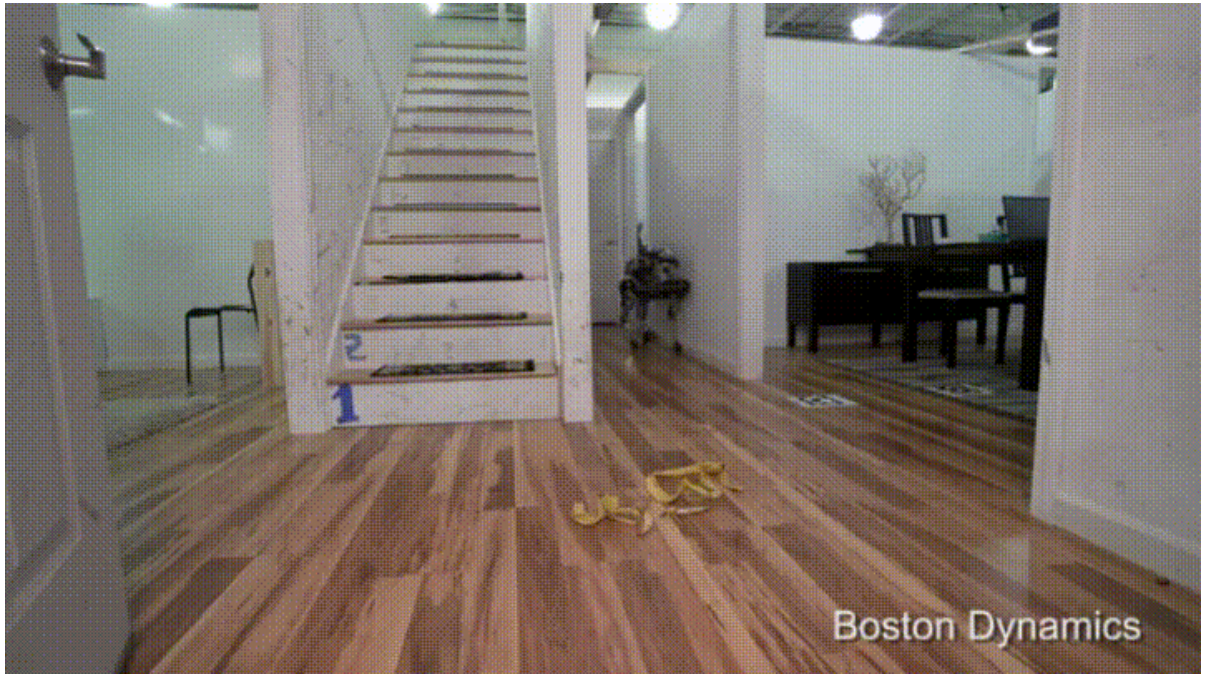
- AlphaGo
  - Teach Go players better strategies
  - Explanation for unorthodox plays, such as move 37
- Rubik's Cube
  - Teach people to solve the Rubik's cube
  - Group Theory
    - Symmetry
    - Conjugate Triplets
  - Ongoing research at UofSC





Open Problems: Safety and Ethics

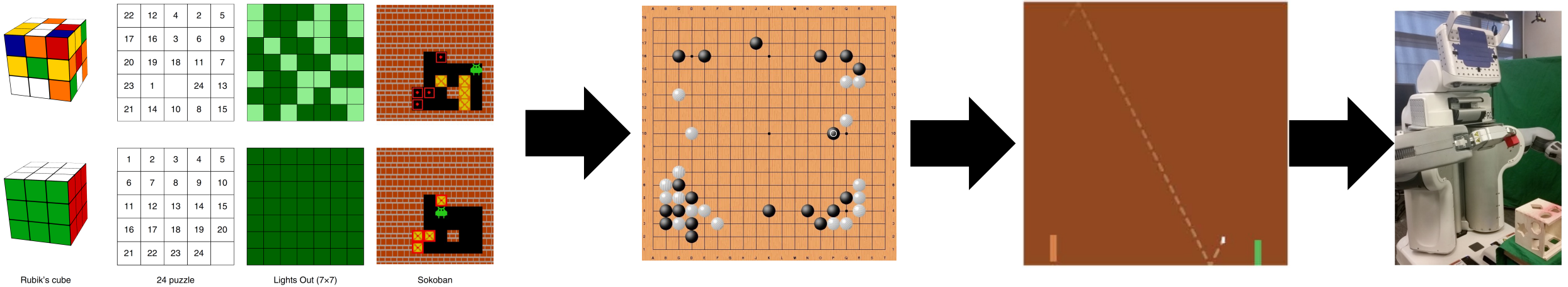
- With reinforcement learning, we learn from our own experience
- How to avoid actions that are too dangerous
  - Self-driving cars
  - Elderly care
  - Automated chemistry



This Photo by Unknown Author is licensed under CC BY-ND

Open Problems: Transfer/Lifelong Learning

- Currently, individual agents are trained for individual tasks
- Transferring knowledge across tasks, especially in extremely different domains, is an open problem



Coding Homework 0

- Setting up the Python environments
- Nothing to turn in, but it is very important to your success in this class!